

RAPPORT TECHNIQUE DE SECURITE

Analyse Statique – Application Android Nickel

com.fpe.comptenickel | Version 2.12.0 | 29 avril 2026

Outil d'analyse	MobSF (Mobile Security Framework)
APK analysé	app.apk (23.2 MB)
Score de sécurité	43 / 100 ⚠ INSUFFISANT
Trackers détectés	3 / 432
Catégorie Play Store	Finance
Analyste	Équipe Sécurité – ESIEE-IT 2025/2026

⚠ Document confidentiel – usage interne uniquement

1. Introduction et contexte

Ce rapport présente les résultats de l'analyse de sécurité statique de l'application Android Nickel (com.fpe.comptenickel), réalisée dans le cadre du projet académique de sécurité mobile – Story 9 du backlog ESIEE-IT 2025/2026.

L'analyse a été conduite à l'aide de MobSF (Mobile Security Framework), un outil d'analyse statique automatisée reconnu dans l'industrie. L'objectif est d'identifier les vulnérabilités présentes dans le binaire APK, d'en évaluer le risque pour une application bancaire et de formuler des recommandations de remédiation.

1.1 Périmètre de l'analyse

- Type d'analyse : Analyse statique (aucune exécution du binaire)
- Outil : MobSF – instance locale
- Date : 29 avril 2026 à 10h35
- Fichier analysé : app.apk – SHA256 : f01f08c8b6e2fe46...
- Application : Nickel – service de compte bancaire grand public

2. Métadonnées de l'application

Propriété	Valeur
Nom	Nickel
Package	com.fpe.comptenickel
Version	2.12.0 (code build : 150)
Taille APK	23.2 MB
SDK minimum	19 (Android 4.4 KitKat)
SDK cible	28 (Android 9 Pie)
Catégorie	Finance
MD5	262caf29e053404...
SHA256	f01f08c8b6e2fe46...
Activité principale	com.fpe.comptenickel.MainActivity
Développeur	NICKEL https://nickel.eu
Score sécurité MobSF	43 / 100 – INSUFFISANT
Trackers détectés	3 sur 432 testés

3. Score de sécurité global

Le score obtenu par l'application Nickel est de 43/100, ce qui est significativement insuffisant pour une application financière manipulant des données bancaires sensibles. À titre de référence, les meilleures pratiques sectorielles recommandent un score supérieur à 70/100 pour ce type d'application.

Score MobSF obtenu	43 / 100
Seuil recommandé (app. financière)	> 70 / 100
Écart constaté	- 27 points

Les principales catégories de risque contribuant à ce score dégradé sont : la politique de signature du certificat (vulnérabilité Janus), la configuration du manifeste Android (trafic en clair, SDK obsolète), les composants exportés sans protection, et l'absence de FORTIFY sur les bibliothèques natives.

4. Trackers tiers détectés

MobSF a identifié 3 SDK tiers sur les 432 patterns analysés. Ces trackers collectent des données comportementales et d'usage qui sont transmises vers des serveurs externes, principalement situés aux États-Unis.

Tracker	Catégorie	Risque pour l'utilisateur
Google CrashLytics	Crash reporting	Transmissions de traces d'erreur pouvant contenir des données contextuelles
Google Firebase Analytics	Analytics	Collecte de données d'usage et de comportement utilisateur
Pushwoosh	Notifications push	Service de push tiers avec stockage potentiel de tokens en dehors de l'UE

Point d'attention RGPD : Pushwoosh est une société dont les serveurs peuvent être situés hors UE. Pour une application financière soumise aux obligations réglementaires françaises et européennes, il convient de vérifier si ce transfert de données est conforme aux exigences du RGPD et à la politique de confidentialité affichée.

5. Vulnérabilités du certificat

L'analyse du certificat de signature révèle deux avertissements importants, dont une vulnérabilité connue pouvant permettre à un attaquant d'installer une version modifiée de l'application sur certains appareils Android.

Sévérité	Problème	Description
WARNING	Vulnérabilité Janus	Signé en v1 uniquement – vulnérable sur Android 5.0 à 8.0. Permet la modification de l'APK sans invalider la signature.
WARNING	Algorithme SHA1withRSA	SHA-1 est considéré cryptographiquement faible et vulnérable aux attaques par collision. Son usage n'est plus recommandé depuis 2017.
INFO	Application signée	L'application est bien signée avec un certificat de signature de code valide.

Détails du certificat : émis par CN=fred, O=squirrel, OU=dsi, ST=idf, L=paris, C=fr. Valide du 2016-06-07 au 2041-06-01. Clé RSA 2048 bits. L'application ne dispose que de la signature v1, rendant inapplicable la protection offerte par les schémas v2 et v3.

6. Vulnérabilités du manifeste Android

6.1 Issues critiques (HIGH)

N°	Sévérité	Problème	Description et impact
1	HIGH	SDK Android obsolète (minSdk=19)	L'application peut être installée sur Android 4.4 (KitKat), une version qui a atteint sa fin de vie en 2019 et cumule des centaines de CVE non corrigées. Cela expose les utilisateurs sur d'anciens appareils à des attaques système.
2	HIGH	Trafic réseau en clair (CleartextTraffic=true)	L'application autorise les connexions HTTP non chiffrées. Un attaquant en position Man-in-the-Middle (réseau Wi-Fi public, hotspot malveillant) peut intercepter et modifier les échanges réseau, compromettant potentiellement des données bancaires.

6.2 Issues moyennes (WARNING)

N°	Sévérité	Composant	Description
3	WARNING	RNDeviceInfo.BroadcastReceiver	Broadcast Receiver non protégé (exported=true) – toute application peut lui envoyer des intents.
4	WARNING	PushAmazonReceiver	Protégé par une permission mais celle-ci n'est pas vérifiée correctement.
5	WARNING	BootReceiver (Pushwoosh)	Protégé par RECEIVE_BOOT_COMPLETED mais la permission n'est pas vérifiée.
6	WARNING	FcmRegistrationService	Service sans protection exposé via un intent-filter – accessible par toute app.
7	WARNING	PushFcmIntentService	Service sans protection exposé via un intent-filter.
8	WARNING	PushwooshSharedDataProvider	Content Provider exporté sans protection – peut exposer des données de configuration.
9	WARNING	FirebaseInstanceIdReceiver	Broadcast Receiver partiellement protégé – la permission n'est pas correctement vérifiée.
10	WARNING	AppMeasurementInstallReferrerReceiver	Protection par permission non vérifiée au niveau requis.

7. Permissions sensibles

7.1 Permissions dangereuses (DANGEROUS)

Android classe certaines permissions comme "dangereuses" car elles donnent accès à des données personnelles ou des fonctions critiques. L'application déclare les deux permissions dangereuses suivantes :

Permission	Niveau	Impact potentiel
android.permission.READ_CONTACTS	DANGEROUS	Accès à l'intégralité du carnet de contacts – justification fonctionnelle à documenter.
android.permission.WRITE_EXTERNAL_STORAGE	DANGEROUS	Écriture sur le stockage externe partagé – peut exposer des fichiers de l'app à d'autres applications.

7.2 Autres permissions notables

Permission	Statut	Commentaire
android.permission.INTERNET	Normal	Accès réseau standard – attendu pour une app bancaire.
android.permission.USE_FINGERPRINT	Dépréciée	API dépréciée depuis Android 9 (API 28) – doit être remplacée par USE_BIOMETRIC.
android.permission.RECEIVE_BOOT_COMPLETED	Normal	Démarrage automatique au boot – utilisé par Pushwoosh pour les notifications.
android.permission.WAKE_LOCK	Normal	Empêche la mise en veille – impact batterie potentiel.

8. Analyse des bibliothèques partagées

156 bibliothèques partagées (.so) ont été analysées. Elles présentent toutes un profil de sécurité identique, avec un point positif majeur (protections mémoire activées) et un point négatif récurrent : l'absence de FORTIFY_SOURCE.

Protection	Statut	Résultat	Description
NX (No-Execute)	True	✓ OK	Pages mémoire marquées non-exécutables – protection contre l'injection de shellcode.
PIE / DSO	True	✓ OK	Code indépendant de la position – rend les attaques ROP difficiles.

Protection	Statut	Résultat	Description
Stack Canary	True	✓ OK	Valeur sentinelle sur la pile – détection de débordements.
Full RELRO	True	✓ OK	Section GOT en lecture seule – protection contre les overwrites de pointeurs.
FORTIFY_SOURCE	False	⚠ WARNING	Aucune vérification contre les buffer overflows glibc – affecte les 156 bibliothèques dont libconceal.so, libcrashlytics.so, libhermes-executor.so.
Symboles strippés	True	i INFO	Symboles de debug retirés – rend le reverse-engineering plus difficile.
RPATH / RUNPATH	None	✓ OK	Aucun chemin de recherche de bibliothèque non sécurisé.

9. Analyse APKiD – Détections spécifiques

APKiD analyse le bytecode et les binaires natifs à la recherche de techniques d'obfuscation, d'anti-debug et d'anti-VM. Les résultats indiquent que l'application intègre des mécanismes de protection contre l'analyse :

Fichier	Type de détection	Détails
classes.dex	Anti-Debug Code	Debug.isDebuggerConnected() – détecte si un débogueur est attaché.
classes.dex	Anti-VM Code	Vérification de Build.FINGERPRINT, MODEL, MANUFACTURER, HARDWARE, BOARD, SERIAL, TAGS, SIM – détecte les émulateurs.
classes.dex	Compilateur	Compilé avec r8 (optimiseur Android standard).
libcrashlytics.so	Anti-hook	Utilisation de syscalls directs – technique pour contourner les hooks système (Frida, Xposed).

Observation : les protections anti-debug et anti-VM constituent une bonne pratique de sécurité pour une application financière. Elles témoignent d'une volonté de protéger l'application contre l'analyse et la manipulation dynamique.

10. Analyse Firebase

L'application communique avec Google Firebase. L'analyse révèle une exposition d'informations de configuration sensibles dans le binaire :

- URL Firebase détectée : application-client-nickel.firebaseio.com
- Firebase Remote Config activé avec les clés de configuration suivantes exposées dans le binaire : APP_VERSION_ADDING, BENEFICIARY_SUPPORT, state, templateVersion
- Ces clés permettent à un analyste de comprendre la logique de feature-flagging de l'application

Test d'accès public réalisé : requête GET sur application-client-nickel.firebaseio.com/.json – la base de données n'est pas exposée publiquement (règles de sécurité Firebase actives).

Point de contrôle Firebase	Résultat	Commentaire
Accès public à la base de données	Non exposée	Règles de sécurité Firebase correctement configurées.
Remote Config exposé dans le binaire	Clés détectées	APP_VERSION_ADDING, BENEFICIARY_SUPPORT, state, templateVersion – information de configuration visible.

11. Serveurs contactés

L'analyse identifie trois domaines contactés par l'application. Un point d'attention notable concerne la résolution géographique du serveur d'API principal :

Domaine	Localisation IP	Adresse IP	Statut
api.nickel.eu	Japon, Tokyo 	23.53.1.137	OK
application-client-nickel.firebaseio.com	USA, Kansas City	34.120.206.254	OK
www.openssl.org	USA, Houston	34.49.79.89	OK

Point d'attention : le domaine api.nickel.eu résout vers une adresse IP localisée à Tokyo, Japon (23.53.1.137). Il est probable que cette résolution passe par un CDN (Akamai ou similaire) dont les nœuds de sortie sont distribués mondialement. Ce point mérite toutefois vérification auprès de l'équipe infrastructure pour s'assurer qu'aucune donnée bancaire ne transite via des serveurs en dehors de l'EEE.

12. Synthèse des risques

Le tableau ci-dessous récapitule l'ensemble des vulnérabilités identifiées, classées par priorité de remédiation :

Priorité	Vulnérabilité	Impact
CRITIQUE	SDK minSdk=19 (Android 4.4)	Exposition à des centaines de CVE non corrigées
CRITIQUE	CleartextTraffic=true	Interception MITM des données bancaires
HAUTE	Vulnérabilité Janus (signature v1 seule)	Modification de l'APK sans invalider la signature
HAUTE	8 composants exportés sans protection	Interaction malveillante depuis des apps tierces

Priorité	Vulnérabilité	Impact
HAUTE	FORTIFY désactivé sur 156 bibliothèques	Risque de buffer overflow non détecté
MOYENNE	Permissions READ_CONTACTS et WRITE_EXTERNAL_STORAGE	Accès non justifié à des données personnelles
MOYENNE	USE_FINGERPRINT dépréciée	Comportement non supporté sur API >= 28
BASSE	Remote Config Firebase exposé dans le binaire	Fuites d'informations de configuration
BASSE	api.nickel.eu → IP Tokyo (CDN à vérifier)	Potentiel transfert hors EEE à confirmer

13. Recommandations

13.1 Actions critiques (à traiter immédiatement)

Action	Mise en œuvre recommandée
Relever minSdk à 24 (Android 7.0)	Modifier android:minSdkVersion="24" dans le AndroidManifest.xml et build.gradle. Cela exclut environ 2% des appareils actifs mais élimine des centaines de CVE.
Désactiver cleartextTraffic	Mettre android:usesCleartextTraffic="false" ou implémenter un Network Security Config restrictif avec <domain includeSubdomains="true"> pour chaque domaine autorisé.

13.2 Actions hautement prioritaires

Action	Mise en œuvre recommandée
Activer signature v2/v3 exclusivement	Dans build.gradle : v1SigningEnabled false, v2SigningEnabled true, v3SigningEnabled true. Corrige la vulnérabilité Janus.
Protéger les composants exportés	Ajouter android:exported="false" sur les 8 composants identifiés, ou définir une permission android:protectionLevel="signature" pour les composants qui doivent rester exportés.
Activer FORTIFY_SOURCE=2	Ajouter -D_FORTIFY_SOURCE=2 dans les flags de compilation du CMakeLists.txt ou Android.mk pour tous les modules natifs.

13.3 Actions de priorité moyenne et basse

Priorité	Action	Recommandation
MOYENNE	Permissions READ_CONTACTS / WRITE_EXTERNAL_STORAGE	Évaluer si ces permissions sont réellement nécessaires. Si oui, documenter la justification. Envisager l'accès granulaire (READ_MEDIA_IMAGES pour Android 13+).

Priorité	Action	Recommandation
MOYENNE	API USE_FINGERPRINT dépréciée	Migrer vers android.permission.USE_BIOMETRIC pour assurer la compatibilité avec Android 9 et supérieur.
BASSE	Remote Config Firebase dans le binaire	Utiliser la minification R8/ProGuard pour obfusquer les noms de clés de configuration sensibles.
BASSE	Localisation IP de api.nickel.eu	Confirmer avec l'équipe infrastructure que les données bancaires ne transitent pas hors de l'EEE. Documenter la configuration CDN.

14. Conclusion

L'analyse statique de l'application Nickel (v2.12.0) révèle un score de sécurité de 43/100, insuffisant pour une application bancaire grand public. Deux vulnérabilités critiques – le support d'Android 4.4 et l'autorisation du trafic réseau en clair – représentent un risque immédiat pour les utilisateurs et doivent être traitées en priorité absolue.

Sur le plan positif, l'application intègre des protections mémoire complètes (NX, PIE, Stack Canary, Full RELRO), des mécanismes anti-debug et anti-VM témoignant d'une conscience sécurité, et les règles de sécurité Firebase sont correctement configurées.

La mise en œuvre des 7 recommandations formulées dans ce rapport permettrait d'amener le score de sécurité au-delà de 70/100 et de réduire significativement la surface d'attaque de l'application. Une analyse dynamique complémentaire (DAST) est recommandée pour valider le comportement réseau réel de l'application et compléter les conclusions de cette analyse statique.

— Fin du rapport —